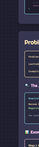

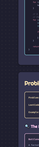

HashMap

Key-Value Pairs | O(1) Lookup

📖 The Diary Analogy

Core Concept:

A HashMap is like a **personal diary** where you record events with specific dates (keys) to easily look them up later.

🎯 Real-World Example

Dec 5	→	Bought a bike
Dec 6	→	Broke my leg
Dec 7	→	Sold the bike

Question: "What happened on Dec 7?" → **Instant Answer:** "Sold the bike"

🔑 Key-Value Pairs

KEY (Identifier)

- **Must be UNIQUE**
- You decide what it is (date, name, character)
- Used to look up data
- Examples: "Dec 7", 'A', "John"

VALUE (Data)

- **Can be DUPLICATED**
- The actual information stored
- Retrieved using the key
- Examples: "Sold bike", 5, [1,2,3]

✅ What HashMap CAN Do

- **Retrieve value** for a specific key → `map.get(key)`
- **Check if key exists** → `map.containsKey(key)`
- **Get size** (number of pairs) → `map.size()`
- **Insert/Update** → `map.put(key, value)`

❌ What HashMap CANNOT Do

Nonsense Questions:

❌ "What happened BEFORE Dec 10?" → HashMap is **unordered**
❌ "What was the LAST entry?" → No order tracking
❌ Range queries → Use TreeMap or arrays instead

💻 Java Syntax

```java
// Declaration (requires Object types, not primitives)
HashMap<Integer, Integer> map = new HashMap<>();

// Insert or Update
map.put(5, 100);  // Key: 5, Value: 100

// Retrieve
int value = map.get(5);  // Returns 100

// Check if key exists
if (map.containsKey(5)) {
    System.out.println("Key 5 exists!");
}

// Get with default value (avoid null issues)
int val = map.getOrDefault(99, 0);  // Returns 0 if key 99 doesn't exist

// Remove a key
map.remove(5);

// Iterate through keys
for (Integer key : map.keySet()) {
    System.out.println(key + " -> " + map.get(key));
}

// Iterate through values
for (Integer value : map.values()) {
    System.out.println(value);
}
```

🎯 When to Use HashMap

Primary Use Case:

Remember **OLD DATA** to answer specific lookup questions later.

Character Frequency
Count occurrences of each character in a string

Lookup & Check
"Have I seen this before?" type problems

Counting & Tracking
Track frequency, first occurrence, last occurrence

Matching & Pairing
Compare two sets of data (Need vs Have)

🔥 String Problems = Character Frequency

In **99% of string problems**, HashMap stores:

Key: Character (e.g., 'A', 'B')
Value: Count (how many times it appears)

Problem 1: First Unique Character

Problem: Find the first character in a string that appears only once. Return its index. If none exist, return -1.

LeetCode: #387

Example: "leetcode" → Index 0 ('l')

🔍 Algorithm

Two-Pass Strategy:

Pass 1: Build frequency map (count all characters)
Pass 2: Scan from left to right, find first char with frequency = 1

📊 Example: "leetcode"

Pass 1: Build Frequency Map

0	1	2	3	4	5	6	7
l	e	e	t	c	o	d	e

l	e	t	c	o	d
1	3	1	1	1	1

Pass 2: Find First with Frequency = 1

0	1	2	3	4	5	6	7
l	e	e	t	c	o	d	e

Check index 0 ('l'): map.get('l') = 1 → FOUND!
Return index 0

💻 Java Code

```java
/**
 * First Unique Character in a String
 * LeetCode 387
 *
 * Strategy: Two-pass
 * - Pass 1: Build frequency map
 * - Pass 2: Find first character with frequency = 1
 *
 * Time: O(n), Space: O(1) - at most 26 letters
 */
public int firstUniqChar(String s) {
    // 1. Create HashMap for character frequency
    HashMap<Character, Integer> map = new HashMap<>();

    // 2. Build the frequency map (Pass 1)
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        // If key exists, increment count; else insert 1
        map.put(c, map.getOrDefault(c, 0) + 1);
    }

    // 3. Find the first unique character (Pass 2)
    for (int i = 0; i < s.length(); i++) {
        if (map.get(s.charAt(i)) == 1) {
            return i; // Return the index
        }
    }

    // 4. If no unique character found
    return -1;
}
```

Problem 2: Ransom Note

Problem: Check if "ransomNote" can be constructed using letters from "magazine". Each letter can only be used once.

LeetCode: #383

Example: ransomNote = "aa", magazine = "aab" → true

🔍 The Job Interview Analogy

Requirement vs Resume:

Ransom Note = Job Requirements (NEED)
Magazine = Candidate's Resume (HAVE)

For every skill needed, check if candidate has it!

📊 Example: "AAB" from "BAAXX"

Step 1: Build Both Maps

NEED (Ransom)

A	B
2	1

HAVE (Magazine)

B	A	X
1	2	2

Step 2: Verification (Check Each NEED)

Check 'A':
Need: 2 | Have: 2
✓ Have ≥ Need? YES

Check 'B':
Need: 1 | Have: 1
✓ Have ≥ Need? YES

✓ RESULT: TRUE
All requirements satisfied!

💻 Java Code

```java
/**
 * Ransom Note
 * LeetCode 383
 *
 * Strategy: Build two frequency maps (NEED vs HAVE)
 * Check if HAVE satisfies all NEED requirements
 *
 * Time: O(n + m), Space: O(1) - at most 26 letters
 */
public boolean canConstruct(String ransomNote, String magazine) {
    HashMap<Character, Integer> have = new HashMap<>();
    HashMap<Character, Integer> need = new HashMap<>();

    // 1. Build NEED map (Ransom Note)
    for (char c : ransomNote.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }

    // 2. Build HAVE map (Magazine)
    for (char c : magazine.toCharArray()) {
        have.put(c, have.getOrDefault(c, 0) + 1);
    }

    // 3. Check requirements
    // Iterate ONLY over keys in 'need' (we only care what's required)
    for (char key : need.keySet()) {
        int neededCount = need.get(key);
        int haveCount = have.getOrDefault(key, 0); // 0 if not in magazine

        if (haveCount < neededCount) {
            return false; // Not enough characters
        }
    }

    return true;
}
```

Problem 3: Maximum Number of Balloons

Problem: Given a string, calculate how many times you can form the word "balloon".

LeetCode: #1189

Example: "loonbalxballpoon" → 2

🔍 The Factory/Slowest Worker Analogy

Bottleneck Concept:

A factory is only as fast as its **slowest worker**!

If you have parts for 100 cars but only 8 engines → You can only make **8 cars**.

🎯 The Math

For each letter in "balloon":

Possible Words = Have / Need

Example: If you have 6 'L's and need 2 'L's per word:
→ 6 / 2 = **3 words possible**

Answer = MINIMUM of all ratios

📊 Example: "BBAALLOOOOONN"

Target: "balloon" Analysis

Letter	Have	Need	Ratio (Have/Need)	Max Possible
B	2	1	2 / 1	2
A	2	1	2 / 1	2
L	2	2	2 / 2	1 ← BOTTLENECK
O	5	2	5 / 2	2
N	2	1	2 / 1	2

Minimum = 1
We can form the word "balloon" 1 time

💻 Java Code

```java
/**
 * Maximum Number of Balloons
 * LeetCode 1189
 *
 * Strategy: Calculate Have/Need ratio for each letter
 * Answer = Minimum ratio (bottleneck)
 *
 * Time: O(n), Space: O(1)
 */
public int maxNumberOfBalloons(String text) {
    HashMap<Character, Integer> have = new HashMap<>();
    HashMap<Character, Integer> need = new HashMap<>();

    // 1. Build NEED map for "balloon"
    String target = "balloon";
    for (char c : target.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }
    // Result: {b:1, a:1, l:2, o:2, n:1}

    // 2. Build HAVE map from input text
    for (char c : text.toCharArray()) {
        have.put(c, have.getOrDefault(c, 0) + 1);
    }

    int ans = Integer.MAX_VALUE;

    // 3. Calculate minimum ratio (bottleneck)
    for (char key : need.keySet()) {
        int neededCount = need.get(key);
        int haveCount = have.getOrDefault(key, 0);

        // How many times can this specific letter support the word?
        int times = haveCount / neededCount;

        // Answer is limited by the slowest worker
        ans = Math.min(ans, times);
    }

    return ans;
}
```

Problem 4: Longest Palindrome

Problem: Find the length of the longest palindrome that can be built with the given letters (case sensitive).

LeetCode: #409

Example: "abccccdd" → 7 ("dccaccd" or similar)

🔍 Palindrome Rules

EVEN Counts

✓ Use ALL characters
Example: 4 'C's → CCCC
(2 pairs, perfect for palindrome)

ODD Counts

⚠ Use (Count - 1)
Example: 5 'A's → Use 4, leave 1
(2 pairs used, 1 leftover)

The Center Rule:

If you had **any odd counts**, you can place **ONE** character in the center!
This adds **+1** to the final length.

📊 Example: "AAABBBCCCCD"

Step-by-Step Processing

Map: {A:3, B:3, C:4, D:1}

1. 'C' (4): EVEN
   Use all 4 → Total = 4

2. 'A' (3): ODD
   Use (3-1) = 2 → Total = 6
   hasOdd = true (1 'A' remains)

3. 'B' (3): ODD
   Use (3-1) = 2 → Total = 8
   hasOdd = true (1 'B' remains)

4. 'D' (1): ODD
   Use (1-1) = 0 → Total = 8
   hasOdd = true (1 'D' remains)

Final Adjustment:
Had odd counts? YES → Add 1 for center
Final Length = 8 + 1 = 9

💻 Java Code

```java
/**
 * Longest Palindrome
 * LeetCode 409
 *
 * Strategy:
 * - Even counts: Use all
 * - Odd counts: Use (count-1), track that we had odd
 * - If had any odd: Add 1 for center
 *
 * Time: O(n), Space: O(1)
 */
public int longestPalindrome(String s) {
    HashMap<Character, Integer> map = new HashMap<>();

    // 1. Build Frequency Map
    for (char c : s.toCharArray()) {
        map.put(c, map.getOrDefault(c, 0) + 1);
    }

    int result = 0;
    boolean hasOdd = false;

    // 2. Iterate through frequencies
    for (int val : map.values()) {
        if (val % 2 == 0) {
            // Even: Use all characters
            result += val;
        } else {
            // Odd: Use (val - 1) to make it even
            result += (val - 1);
            hasOdd = true; // Mark that we have odd characters
        }
    }

    // 3. If we had any odd characters, we can put ONE in the center
    if (hasOdd) {
        return result + 1;
    } else {
        return result;
    }
}
```

📝 Pattern Summary

The Core Question:

"Do I need to **remember data** for later lookup?"

🎯 Common Patterns

Problem Type	Key	Value
Character Frequency	Character	Count
First Unique Character	Character	Count
Two Sum	Number	Index
Group Anagrams	Sorted String	List of Strings
Word Pattern	Character/Word	Word/Character
Longest Consecutive	Number	Boolean (exists)

💡 Key Takeaways

- **String Problems:** 99% use Character → Count mapping
- **Two Maps:** Often compare NEED vs HAVE
- **getOrDefault():** Avoid null checks, cleaner code
- **Iterate keySet():** Only check what you need
- **Time Complexity:** O(1) lookup, O(n) to build map

🚀 Template Code

```java
// Standard Frequency Map Template
HashMap<Character, Integer> map = new HashMap<>();

// Build map
for (char c : s.toCharArray()) {
    map.put(c, map.getOrDefault(c, 0) + 1);
}

// Use map
for (char key : map.keySet()) {
    int count = map.get(key);
    // Your logic here
}
```